Truth UI Context

Document: `ui-context.md`
Project: Truth
Purpose: Define the user interface's role, boundaries, interaction principles, and implementation requirements.

---

1. Purpose of the Interface

Truth is a software system designed to help a person understand their current situation, connect relevant information, reason about possibilities, and move toward intended outcomes.

The user interface is the controlled interaction layer between the user and the Truth system. It MUST make Truth understandable and usable without requiring the user to understand its internal technical architecture.

The interface is responsible for presenting information, collecting user input, communicating system status, displaying relevant results, and requesting authorization for actions where required. The interface MUST NOT independently duplicate the Kernel's reasoning, memory management, state transitions, or execution logic.

---

2. Current Implementation Versus Future Vision

Truth's interface must reflect the actual state of the repository. Before implementing any UI feature, the coding agent MUST inspect the existing codebase to establish:

- Whether a frontend already exists.
- Which frameworks and libraries are installed.
- Which API endpoints and backend capabilities are available.
- Which interactions are implemented and tested.
- Which proposed features remain conceptual.

Do not assume a particular frontend framework, mobile platform, component library, or design system without evidence from the repository or an approved task. If the current repository contains only a backend prototype, do not invent an existing interface or silently introduce a large frontend architecture.

---

3. Core Interface Principles

Every interface decision MUST follow these principles:

- **3.1 Clarity:** Simple language, clear presentation of what Truth knows, proposes, and enables next.
- **3.2 Intentional interaction:** Every prominent control must perform its stated action.
- **3.3 Consistency:** Reuse established visual patterns, terminology, and components.
- **3.4 User control:** The user remains in control of personal information, permissions, consequential actions, and system settings.
- **3.5 Honest system status:** Distinguish between requested, in-progress, completed, and failed operations. Never display success before backend confirmation.
- **3.6 Progressive complexity:** Present necessary information first; reveal advanced configuration when relevant.

---

4. Relationship Between UI and Kernel

Established execution flow:
`Input` → `Observation` → `Memory Retrieval` → `State` → `Reasoning` → `Response` → `Commit`

Responsibilities of the UI

- Collect user input and display conversations/system responses.
- Present relevant context, retrieved information, and conclusions supplied by backend.
- Display loading, success, warning, and failure states.
- Collect user preferences and permission decisions; show proposed actions for confirmation.

Responsibilities Outside the UI

- Determine authoritative state, write directly to internal memory stores bypassing approved interfaces, invent/modify conclusions from reasoning, treat displayed text as proof of success, or bypass authentication/authorization.

---

5. Primary Interaction Model

Truth supports a natural interaction pattern across stages: **Input** → **Processing** → **Response** → **Context** → **Decision** → **Execution** → **Confirmation**.

---

6. Interface Architecture

Maintain logical separation between presentation components, feature components, application services, state management, and design system elements without creating unnecessary overhead.

---

7. Visual Design Direction

Prioritize legibility, calm visual hierarchy, consistent spacing/typography, useful feedback, responsive behavior, and a coherent visual language without decorative complexity.

---

8. Responsive and Mobile First Considerations

Support established platform environments, prioritize essential workflows on narrow screens, ensure touch-friendly controls, avoid unnecessary horizontal scrolling, and handle loading/error/empty states gracefully.

---

9. Conversation and Input Experience

Distinguish user messages from system responses, provide clean input/submission behavior, show processing status, handle errors close to the action, and avoid displaying invented memory or unsupported explanations.

---

10. Goals, Progress, and Personal Context

Base features on implemented data models and approved backend interfaces. Clearly distinguish between user goals, proposed plans, accepted/in-progress tasks, and verified progress measurements. Never fabricate progress percentages or personal history.

---

11. Actions, Permissions, and External Applications

Make the difference between a suggestion and an executed action clear. For consequential operations, present the relevant action, target, expected effect, and permission requirements before execution.

---

12. Privacy and Local Operation

Support local operation and user control by avoiding unnecessary data collection, explaining permissions clearly, and accurately reflecting storage and synchronization behavior.

---

13. Accessibility and Interaction Quality

Use semantic controls, ensure keyboard navigation and visible contrast, provide actionable validation messages, and support assistive technologies.

---

14. UI State and Failure Handling

Support defined interaction states (**Idle**, **Loading**, **Success**, **Empty**, **Error**, **Unavailable**, **Pending approval**, **Cancelled**) and avoid endless loading indicators or misleading empty results.

---

15. UI Testing and Verification

Verify correct rendering, control behavior, state handling, API communication, permission boundaries, responsive behavior, and accessibility using established repository tools.

---

16. Implementation Rules for the Coding Agent

Inspect the repository, identify acceptance criteria and backend contracts, implement the smallest coherent change, test interactions, and report outcomes honestly without inventing endpoints or mock data.

---

17. Definition of Done

A UI change is complete when it satisfies the approved task, follows design conventions, represents backend status honestly, handles states/permissions correctly, runs tests, and updates documentation.

---

18. Relationship to Other Truth Instructions

Governed alongside `project-overview.md`, `architecture.md`, `ai-workflow-rules.md`, and `code-standards.md`. Kernel contracts take precedence over UI convenience.